

TECHNICAL DOCUMENTATION

AIOps Insurance Document Classifier

Architecture, data flows, ML pipeline design, and system
internals

PROJECT	DATE	STACK	STATUS
eliteclown/aiops	June 2026	Python · Streamlit · scikit-learn	v1 — Initial Release

AI Ops — Insurance Document Classifier: Technical Documentation

A browser-based machine-learning application that automatically classifies insurance documents by category and subcategory using several interchangeable ML algorithms.

Table of Contents

1. [Project Overview](#)
2. [Technology Stack](#)
3. [High-Level Architecture](#)
4. [Module Breakdown](#)
5. 4.1 [Entry Point](#) — `app.py`
6. 4.2 [Document Processor](#) — `src/document_processor.py`
7. 4.3 [Feature Extractor](#) — `src/feature_extractor.py`
8. 4.4 [Classifier](#) — `src/classifier.py`
9. 4.5 [Storage](#) — `src/storage.py`
10. 4.6 [Defaults](#) — `src/defaults.py`
11. [Data Flows](#)
12. 5.1 [Training Flow](#)
13. 5.2 [Prediction Flow](#)
14. [ML Pipeline Architecture](#)
15. [Data Model](#)
16. [UI Application Structure](#)
17. [Directory Structure](#)
18. [Configuration & Deployment](#)
19. [Feature Engineering Deep Dive](#)
20. [Classification Algorithms Compared](#)
21. [Error Handling & Edge Cases](#)

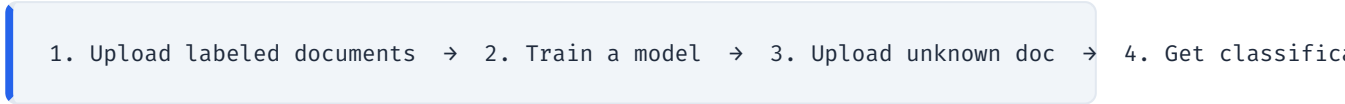
1. Project Overview

AIOPS is a self-contained **Insurance Document Classification System** built with Python and Streamlit. It solves a common problem in the insurance industry: incoming documents (PDFs, Word files, spreadsheets, scanned images) must be routed to the correct processing queue based on their type. Manual sorting is slow and error-prone; this application lets a small team train a custom ML model on their own labeled documents and immediately use it to classify new ones — with no external APIs, no cloud dependencies, and no data leaving the local machine.

What it does

Capability	Detail
Document ingestion	Reads PDF, DOCX/DOC, XLSX/XLS, and raster images (PNG, JPG, TIFF, BMP, GIF)
Text extraction	Native parsing for office formats; OCR (Tesseract) for scanned images
Hierarchical labeling	Two-level taxonomy: Category → Subcategory (e.g., Claims → Medical Records)
Model training	Four interchangeable scikit-learn algorithms with cross-validation
Prediction	Top-5 ranked predictions with probability scores per document
Explainability	Confidence indicator, class distribution chart, confusion matrix
Persistence	Training data in JSON; trained model serialized via joblib

Core User Workflow



2. Technology Stack

```
Language:      Python 3.14+
UI Framework:  Streamlit ≥ 1.35
ML:            scikit-learn ≥ 1.4 (pipelines, TF-IDF, algorithms, cross-validation)
Numerics:      NumPy ≥ 1.26 · pandas ≥ 2.0
Document I/O:  pdfplumber ≥ 0.10 · python-docx ≥ 1.1 · openpyxl ≥ 3.1
Image OCR:     Pillow ≥ 10.0 · pytesseract ≥ 0.3 (+ system Tesseract binary)
Serialization: joblib ≥ 1.3
Visualization: matplotlib ≥ 3.8 · seaborn ≥ 0.13
```

3. High-Level Architecture

The system is a **monolithic single-process Streamlit application**. There is no separate backend API, no database server, and no message queue. All components run in-process; state is held in Streamlit's session state (in-memory) and on disk (JSON + joblib files).

ARCHITECTURE DIAGRAM (SEE RENDERED MARKDOWN FOR INTERACTIVE VERSION)

```
graph TD
    subgraph Browser["User Browser"]
        UI["Streamlit Web UI\n(app.py)"]
    end

    subgraph App["Streamlit Process"]
        direction TB
        SS["Session State\n(classifier, text buffers)"]

        subgraph SRC["src/ package"]
            DP["document_processor\nText Extraction"]
            FE["feature_extractor\nText Preprocessing"]
            CL["classifier\nInsuranceClassifier"]
            ST["storage\nJSON CRUD"]
            DF["defaults\nBuilt-in Taxonomy"]
        end
    end

    subgraph Disk["Local Filesystem"]
        JSON["data/training_data.json"]
        MDL["models/classifier.joblib"]
    end

    subgraph Libs["Third-party Libraries"]
        SKL["scikit-learn\nPipelines & Algorithms"]
        OCR["Tesseract OCR\n(system binary)"]
        PDF["pdfplumber"]
        DOC["python-docx"]
        XLS["openpyxl / pandas"]
    end

    UI <--> |"HTTP (localhost)"| App
    UI --> DP
    DP --> PDF
    DP --> DOC
    DP --> XLS
    DP --> OCR
    DP --> FE
    FE --> CL
    CL --> SKL
    CL --> |"save/load"| MDL
    ST --> |"read/write"| JSON
    DF --> ST
    SS <--> UI
```

4. Module Breakdown

4.1 Entry Point — `app.py`

`app.py` is the **orchestration layer**. It owns the Streamlit page layout and wires all `src/` modules together. It never contains business logic; it delegates everything to the appropriate module.

Responsibilities: - Configure Streamlit page (`set_page_config`) - Initialize session state (load classifier from disk on first run) - Render the three-page navigation via `st.sidebar` - Handle all user interactions (file uploads, button clicks, selects) - Display results, charts, and tables

Key globals:

```
MODEL_PATH = "models/classifier.joblib" # serialized model location
```

Pages rendered:

Page	Sidebar Label	Purpose
Train	Train	Upload + label documents, manage dataset, trigger training
Test / Predict	Test / Predict	Upload a new document, run classification, view ranked results
Model Info	Model Info	Dashboard with metrics, class distribution chart, confusion matrix

4.2 Document Processor — `src/document_processor.py`

Handles **text extraction** from all supported file formats. Acts as a format-agnostic adapter: callers only use `extract_text(bytes, filename)` and receive a plain string regardless of the source format.

Supported formats:

```
SUPPORTED_EXTENSIONS = {
    ".pdf", ".docx", ".doc", ".xlsx", ".xls",
    ".png", ".jpg", ".jpeg", ".tiff", ".tif", ".bmp", ".gif",
}
```

Internal dispatch:

ARCHITECTURE DIAGRAM (SEE RENDERED MARKDOWN FOR INTERACTIVE VERSION)

flowchart LR

```
A["extract_text(bytes, filename)"] → B{file extension}
B -- ".pdf" → C["extract_text_from_pdf()\nnpdfplumber"]
B -- ".docx / .doc" → D["extract_text_from_docx()\npython-docx\n(paragraphs + tables)"]
B -- ".xlsx / .xls" → E["extract_text_from_xlsx()\nopenpyxl / pandas\n(all sheets → text)"]
B -- "image formats" → F["extract_text_from_image()\nPillow + pytesseract OCR"]
C → G["raw text string"]
D → G
E → G
F → G
```

DOCX table handling: table cells are joined with `" | "` separators and appended after paragraph text, ensuring tabular data in claims forms or policies is not silently dropped.

XLSX handling: every sheet is iterated; column headers and all non-empty cell values are linearized into a `Sheet: <name>\ncol1 | col2 | ... \nval1 | val2 | ...` text block.

OCR dependency: `extract_text_from_image` requires the system-level Tesseract binary. If it is missing the function raises a `RuntimeError` with installation instructions for macOS, Ubuntu, and Windows.

4.3 Feature Extractor — [src/feature_extractor.py](#)

Converts raw extracted text into the feature string fed to the ML pipeline. Three functions:

Function	Input	Output	Purpose
<code>preprocess_text(text)</code>	raw str	normalized str	lowercase, strip non-alphanumeric chars, collapse whitespace
<code>prepare_single_text(text, keywords)</code>	raw str + keyword list	feature str	preprocess + append keywords repeated ×3 for TF boost
<code>prepare_training_texts(samples)</code>	list of sample dicts	list of feature strs	batch version of <code>prepare_single_text</code> for training

Keyword boosting mechanism:

```
kw_boost = (kw_block + " ") * 3
return f"{processed} {kw_boost}".strip()
```

When a user marks a document with keywords like `"deductible, claim number"`, those tokens are appended three times to the feature string. Because TF-IDF weighs term frequency, this artificially inflates the importance of user-provided signals without requiring a custom vectorizer.

4.4 Classifier — `src/classifier.py`

The **ML engine**. Wraps scikit-learn pipelines behind a clean domain interface.

INSURANCECLASSIFIER CLASS

ARCHITECTURE DIAGRAM (SEE RENDERED MARKDOWN FOR INTERACTIVE VERSION)

```
classDiagram
    class InsuranceClassifier {
        +str classifier_name
        +Pipeline pipeline
        +bool is_trained
        +float training_accuracy
        +ndarray cv_scores
        +list classes_
        +int n_samples_trained
        +train(texts, compound_labels)
        +predict(text) dict
        +predict_top_n(text, n) list[dict]
        +save(path)
        +load(path)$ InsuranceClassifier
        -_get_confidence(text) float
        -_parse_result(compound, confidence) dict
        -_require_trained()
    }
```

PIPELINE CONSTRUCTION — `_BUILD_PIPELINE(CLASSIFIER_NAME)`

Each algorithm is implemented as a two-step scikit-learn `Pipeline` (vectorizer → estimator):

ARCHITECTURE DIAGRAM (SEE RENDERED MARKDOWN FOR INTERACTIVE VERSION)

```
graph LR
    subgraph "Logistic Regression"
        A1["TF-IDF\n(1-2 grams, 10k features,\nsublinear_tf=True)"] --> B1["LogisticRegression\n(alpha=0.5)"]
    end
    subgraph "Random Forest"
        A2["TF-IDF\n(1-2 grams, 5k features,\nsublinear_tf=True)"] --> B2["RandomForestClassifier\n(n_estimators=100)"]
    end
    subgraph "SVM Linear"
        A3["TF-IDF\n(1-2 grams, 10k features,\nsublinear_tf=True)"] --> B3["CalibratedClassifierCV\n(linear_svm.LinearSVC())"]
    end
    subgraph "Naive Bayes"
        A4["CountVectorizer\n(1-2 grams, 10k features)"] --> B4["MultinomialNB\n(alpha=0.5)"]
    end
```

SVM calibration: `LinearSVC` does not natively expose `predict_proba`. It is wrapped in `CalibratedClassifierCV` (Platt scaling, `cv=3`) so probability scores are available for confidence display and top-N ranking.

Naive Bayes vectorizer: `MultinomialNB` requires non-negative integer input, so `CountVectorizer` (raw term counts) is used instead of TF-IDF for this algorithm only.

TRAINING AND CROSS-VALIDATION

ARCHITECTURE DIAGRAM (SEE RENDERED MARKDOWN FOR INTERACTIVE VERSION)

```
flowchart TD
    A["train(texts, labels)"] --> B["_build_pipeline()"]
    B --> C["pipeline.fit(texts, labels)"]
    C --> D{"min class\ncount ≥ 2?"}
    D -- Yes --> E["cv_folds = min(5, min_class_count)"]
    E --> F["cross_val_score(cv_folds, scoring='accuracy')"]
    F --> G["training_accuracy = cv_scores.mean()"]
    D -- No --> H["pipeline.predict(texts)"]
    H --> I["training_accuracy = correct / total"]
    G --> J["save metadata to self"]
    I --> J
```

The number of cross-validation folds is capped at `min(5, min_class_count)` so that classes with few examples never produce empty test folds.

PREDICTION OUTPUT SCHEMA

```
{
  "category":      "Claims",
  "subcategory":   "Medical Records",
  "compound_label": "Claims|Medical Records",
  "confidence":    0.87,           # None if algorithm has no predict_proba
}
```

4.5 Storage — `src/storage.py`

A **flat-file JSON store** for training samples. All reads and writes go through this module; `app.py` and other modules never touch the filesystem directly.

File location: `data/training_data.json` (relative to the project root, auto-created on first write)

Public API:

Function	Description
<code>load_training_data()</code>	Read and return all samples as a list of dicts
<code>add_training_sample(text, filename, category, subcategory, keywords)</code>	Append one sample, write to disk, return the new sample
<code>delete_training_sample(sample_id)</code>	Remove a sample by UUID, write to disk
<code>clear_all_training_data()</code>	Overwrite the file with an empty list
<code>get_all_categories(include_defaults)</code>	Union of stored categories and <code>DEFAULT_CATEGORIES</code>
<code>get_subcategories_for(category, include_defaults)</code>	Union of stored subcategories and <code>DEFAULT_SUBCATEGORIES[category]</code>

4.6 Defaults — `src/defaults.py`

Provides the **built-in insurance taxonomy** so new users see a useful starting point without any training data.

6 default categories, each with 6–8 subcategories:

Policy	→ Auto Policy, Home/Property Policy, Health Policy, Life Policy, Commercial/Business Policy, Liability Policy, Workers Compensation, Other
Claims	→ Claim Form, Medical Records, Police/Incident Report, Adjuster Report, Settlement Agreement, Proof of Loss, Damage Photos, Other
Underwriting	→ Application Form, Risk Assessment, Inspection Report, Loss History/Loss Run, Questionnaire, Other
Financial	→ Premium Invoice, Payment Receipt, Loss Run Report, Financial Statement, Audit Report, Other
Legal	→ Endorsement, Rider/Amendment, Exclusion Notice, Contract/Agreement, Power of Attorney, Subrogation Letter, Other
Correspondence	→ Cancellation Notice, Renewal Notice, Demand Letter, Coverage Update, Declination Letter, Other

These defaults are merged with user-created categories at runtime

(`get_all_categories` , `get_subcategories_for`), so custom categories coexist with built-in ones without any extra configuration.

5. Data Flows

5.1 Training Flow

ARCHITECTURE DIAGRAM (SEE RENDERED MARKDOWN FOR INTERACTIVE VERSION)

```
sequenceDiagram
    actor User
    participant UI as app.py (UI)
    participant DP as document_processor
    participant FE as feature_extractor
    participant ST as storage
    participant CL as InsuranceClassifier
    participant Disk as Filesystem

    User->>UI: Upload document + select category/subcategory + keywords
    UI->>DP: extract_text(bytes, filename)
    DP->>UI: raw text string
    UI->>UI: show preview, collect labels
    User->>UI: Click "Add to Training Set"
    UI->>ST: add_training_sample(text, filename, category, subcategory, keywords)
    ST->>Disk: write data/training_data.json
    ST->>UI: sample dict

    Note over User,Disk: Repeat for each labeled document

    User->>UI: Select algorithm + Click "Train Model"
    UI->>ST: load_training_data()
    ST->>Disk: read data/training_data.json
    ST->>UI: list of samples
    UI->>FE: prepare_training_texts(samples)
    FE->>UI: list of feature strings
    UI->>CL: InsuranceClassifier(algo).train(texts, compound_labels)
    CL->>CL: _build_pipeline() → pipeline.fit()
    CL->>CL: cross_val_score() if enough samples
    CL->>UI: training_accuracy, cv_scores
    UI->>CL: save(MODEL_PATH)
    CL->>Disk: write models/classifier.joblib
    UI->>User: Display accuracy + CV scores
```

5.2 Prediction Flow

ARCHITECTURE DIAGRAM (SEE RENDERED MARKDOWN FOR INTERACTIVE VERSION)

```
sequenceDiagram
    actor User
    participant UI as app.py (UI)
    participant DP as document_processor
    participant FE as feature_extractor
    participant CL as InsuranceClassifier
    participant Disk as Filesystem

    Note over UI,Disk: On app startup, classifier is auto-loaded from disk
    UI->>Disk: load models/classifier.joblib (if exists)
    Disk->>UI: InsuranceClassifier instance

    User->>UI: Upload document to classify
    UI->>DP: extract_text(bytes, filename)
    DP->>UI: raw text string
    User->>UI: (Optional) Enter keywords + Click "Classify"
    UI->>FE: prepare_single_text(text, keywords)
    FE->>UI: preprocessed feature string
    UI->>CL: predict_top_n(feature_string, n=5)
    CL->>CL: pipeline.predict_proba([text])
    CL->>CL: argsort probabilities descending
    CL->>CL: _parse_result() for each top-N
    CL->>UI: list of {category, subcategory, compound_label, confidence}
    UI->>User: Display ranked results with confidence bar
```

6. ML Pipeline Architecture

Every algorithm shares the same two-stage pipeline structure:

ARCHITECTURE DIAGRAM (SEE RENDERED MARKDOWN FOR INTERACTIVE VERSION)

```
graph LR
    subgraph Input
        T["Preprocessed text string\n(lowercase, alphanumeric,\nkeywords boosted ×3)"]
    end

    subgraph "Stage 1: Vectorization"
        V["Vectorizer\n\nTF-IDF (LR, RF, SVM)\nor\nCountVectorizer (NB)\n• unigrams + bigrams"]
    end

    subgraph "Stage 2: Classification"
        C["Estimator\n\nLogistic Regression\nRandom Forest\nLinearSVC (calibrated)\nMultinomial"]
    end

    subgraph Output
        P["Probability vector\n[p_class1, p_class2, ...]"]
        L["Predicted compound_label\n'Category|Subcategory'"]
    end

    T --> V --> C --> P
    C --> L
```

Compound label encoding: The two-level taxonomy (`Category|Subcategory`) is flattened into a single string label (`"Claims|Medical Records"`) before training. After prediction, `_parse_result` splits on `"|"` to recover both levels. This lets any standard single-label classifier handle the hierarchical taxonomy without modification.

7. Data Model

Training Sample (JSON record)

```
{
  "id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "filename": "claim_form_q1.pdf",
  "text": "patient name john smith date of service 2024 01 15 ... ",
  "category": "Claims",
  "subcategory": "Medical Records",
  "keywords": ["medical", "claim number", "diagnosis"],
  "compound_label": "Claims|Medical Records",
  "timestamp": "2026-06-22T14:23:01.456789"
}
```

Serialized Model (models/classifier.joblib)

```
{
  "pipeline": Pipeline,          # fitted sklearn Pipeline object
  "classifier_name": str,         # e.g. "Logistic Regression"
  "is_trained": bool,
  "training_accuracy": float,    # mean CV accuracy or training accuracy
  "cv_scores": np.ndarray,       # per-fold scores, or None
  "classes_": list[str],         # all compound labels seen during training
  "n_samples_trained": int,
}
```

Entity-Relationship Overview

ARCHITECTURE DIAGRAM (SEE RENDERED MARKDOWN FOR INTERACTIVE VERSION)

```
erDiagram
    TRAINING_SAMPLE {
        uuid id PK
        string filename
        text text
        string category
        string subcategory
        string compound_label
        json keywords
        datetime timestamp
    }

    CATEGORY {
        string name PK
    }

    SUBCATEGORY {
        string name PK
        string category FK
    }

    MODEL {
        string path PK
        string classifier_name
        float training_accuracy
        json cv_scores
        json classes_
        int n_samples_trained
        blob pipeline
    }

    TRAINING_SAMPLE }o--|| CATEGORY : "belongs to"
    TRAINING_SAMPLE }o--|| SUBCATEGORY : "belongs to"
    SUBCATEGORY }o--|| CATEGORY : "child of"
    MODEL ||--o{ TRAINING_SAMPLE : "trained on"
```

8. UI Application Structure

Page Layout

ARCHITECTURE DIAGRAM (SEE RENDERED MARKDOWN FOR INTERACTIVE VERSION)

```
graph TD
    APP["app.py\nStreamlit App"]

    APP --> SIDEBAR["Sidebar\n• App title\n• Model status badge\n• Training sample count\n• Page"]

    APP --> P1["Page: Train\n(two-column layout)"]
    APP --> P2["Page: Test / Predict\n(two-column layout)"]
    APP --> P3["Page: Model Info\n(single-column dashboard)"]

    P1 --> L1["Left column\n• File uploader\n• Text preview (expander)\n• Category selector / a"]
    P1 --> R1["Right column\n• Dataset summary metrics (3 columns)\n• Scrollable sample table\n"]

    P2 --> L2["Left column\n• File uploader\n• Extracted text preview"]
    P2 --> R2["Right column\n• Keywords input\n• Classify button\n• Category + Subcategory metr"]

    P3 --> M3["Top metrics row\n(Samples / Classes / Algorithm / Accuracy)"]
    P3 --> CV3["CV score info box"]
    P3 --> CH3["Horizontal bar chart\n(samples per class)"]
    P3 --> CM3["Confusion matrix heatmap\n(seaborn)"]
    P3 --> TBL3["Full training data table\n+ CSV download button"]
```

Session State

ARCHITECTURE DIAGRAM (SEE RENDERED MARKDOWN FOR INTERACTIVE VERSION)

```
stateDiagram-v2
    [*] --> Init: App starts
    Init --> NoModel: classifier.joblib missing
    Init --> ModelLoaded: classifier.joblib found

    NoModel --> Training: user adds samples + trains
    ModelLoaded --> Training: user retrains
    Training --> ModelLoaded: train() + save() succeed

    ModelLoaded --> Predicting: user uploads doc on Test page
    Predicting --> ModelLoaded: prediction displayed

    state Init {
        [*] --> CheckDisk
        CheckDisk --> LoadClassifier: file exists
        CheckDisk --> NullClassifier: file missing
    }
```

Session state keys:

Key	Type	Description
<code>classifier</code>	<code>InsuranceClassifier \</code> <code>None</code>	Active trained model
<code>train_text</code>	<code>str \</code> <code>None</code>	Extracted text from training upload
<code>train_filename</code>	<code>str \</code> <code>None</code>	Filename of training upload
<code>test_text</code>	<code>str \</code> <code>None</code>	Extracted text from prediction upload

9. Directory Structure

```

aiops/
├── app.py                # Streamlit entry point & UI orchestration
├── requirements.txt      # Python dependency pinning
├── src/                  # Core business logic package
│   ├── __init__.py
│   ├── classifier.py    # InsuranceClassifier + pipeline factory
│   ├── defaults.py      # Built-in insurance taxonomy
│   ├── document_processor.py # Multi-format text extraction
│   ├── feature_extractor.py # Text preprocessing + keyword boosting
│   └── storage.py        # JSON-based training data CRUD
├── data/
│   └── training_data.json # Created automatically on first write
│                           # Labeled training samples (flat JSON array)
└── models/
    └── classifier.joblib # Created automatically on first train
                          # Serialized sklearn Pipeline + metadata

```

10. Configuration & Deployment

Running locally

```
pip install -r requirements.txt
# Install Tesseract for image/OCR support:
#   macOS: brew install tesseract
#   Ubuntu: sudo apt-get install tesseract-ocr
#   Windows: https://github.com/UB-Mannheim/tesseract/wiki

streamlit run app.py
```

Streamlit opens a browser tab at `http://localhost:8501` by default.

Environment requirements

Requirement	Notes
Python 3.14+	Uses <code>X \ Y</code> union type hints natively
Tesseract binary	Only needed if classifying scanned image files
~200 MB disk	scikit-learn + pandas + pdfplumber
No GPU required	All ML is CPU-based
No network access	Fully offline after <code>pip install</code>

Hardcoded paths

Both paths are relative to the project root and constructed with `pathlib.Path`:

```
# app.py
MODEL_PATH = str(Path(__file__).parent / "models" / "classifier.joblib")

# src/storage.py
_DATA_DIR = Path(__file__).parent.parent / "data"
_TRAINING_FILE = _DATA_DIR / "training_data.json"
```

Both directories are created automatically (`mkdir(exist_ok=True)`) on first write, so no manual setup is needed.

11. Feature Engineering Deep Dive

The feature engineering pipeline is intentionally simple but effective for domain-specific document classification:

ARCHITECTURE DIAGRAM (SEE RENDERED MARKDOWN FOR INTERACTIVE VERSION)

```
flowchart TD
    A["Raw extracted text\n(may contain headers, footers,\ntables, OCR artifacts)"]
    B["preprocess_text()\n1. lowercase\n2. strip [^a-z0-9\\s]\n3. collapse whitespace"]
    C["User-provided keywords\n(optional, comma-separated)"]
    D["preprocess_text() per keyword"]
    E["Repeat keyword block ×3\n→ inflates TF weight"]
    F["Concatenate:\nprocessed_text + kw_kw_kw"]
    G["Vectorizer\nTF-IDF or CountVec"]
    H["Feature matrix\n(sparse, n_samples × n_features)"]

    A → B
    C → D → E
    B → F
    E → F
    F → G → H
```

Why keyword boosting works with TF-IDF: TF-IDF scores are proportional to term frequency within a document. By repeating keywords three times in the feature string, those tokens receive a TF multiplier of approximately 3×, making them stronger discriminators without needing to modify the vectorizer or introduce custom weights. This is a lightweight but effective human-in-the-loop signal.

12. Classification Algorithms Compared

Property	Logistic Regression	Random Forest	SVM Linear	Naive Bayes
Vectorizer	TF-IDF	TF-IDF	TF-IDF	CountVectorizer
Max features	10,000	5,000	10,000	10,000
N-grams	(1,2)	(1,2)	(1,2)	(1,2)
Sublinear TF	Yes	Yes	Yes	No
Native <code>predict_proba</code>	Yes	Yes	No (calibrated)	Yes
Parallelism	No	Yes (<code>n_jobs=-1</code>)	No	No
Best for	General baseline, interpretable	Larger datasets, robust	High-dimensional text	Very small datasets
Typical speed	Fast	Slower (200 trees)	Fast	Very fast
Confidence quality	Good	Good	Via Platt scaling (approximate)	Often overconfident

13. Error Handling & Edge Cases

Scenario	Handling
Unsupported file extension	<code>ValueError</code> raised in <code>extract_text</code> , shown as <code>st.error</code>
Tesseract not installed	<code>RuntimeError</code> with installation instructions
Corrupt PDF/DOCX	Exception caught in <code>app.py</code> , shown as <code>st.error</code>
Fewer than 2 training samples	UI warning, Train button disabled
Single-class training data	UI warning; training requires ≥ 2 distinct compound labels
Min class count = 1	Cross-validation skipped; training accuracy used instead
Missing model file on startup	<code>_load_classifier_from_disk</code> returns <code>None</code> ; sidebar shows warning
Corrupt model file	<code>try/except</code> around <code>InsuranceClassifier.load</code> , returns <code>None</code>
Classifier without <code>predict_proba</code>	<code>predict_top_n</code> falls back to <code>predict</code> (returns single result)

14. Extension Points

The codebase has several natural extension points that require minimal changes:

Extension	Where to change	Notes
Add a new ML algorithm	<code>classifier.py: _build_pipeline()</code> + <code>ALGORITHM_NAMES</code> list	One new <code>elif</code> branch
Add a new file format	<code>document_processor.py: SUPPORTED_EXTENSIONS</code> + <code>extract_text()</code>	One new <code>elif</code> and extraction function
Switch to a database backend	<code>storage.py</code> only	Replace JSON read/write with SQLite or PostgreSQL; public API is unchanged
Add new default categories	<code>defaults.py: DEFAULT_CATEGORIES</code> + <code>DEFAULT_SUBCATEGORIES</code>	No other changes needed
REST API wrapper	Wrap <code>classifier.predict</code> and <code>document_processor.extract_text</code> in FastAPI	Core logic is already decoupled from Streamlit
Multi-user support	Replace session state + local files with per-user storage	Requires auth layer and user-scoped file paths
Fine-tuned LLM classifier	Implement a new class with the same <code>train</code> / <code>predict</code> / <code>save</code> / <code>load</code> interface	Drop-in replacement for <code>InsuranceClassifier</code>